

DESARROLLO PARA DISPOSITIVOS INTELIGENTES

PRACTICA 3.1

Configuración Entorno PWA

Primer despliegue en Smart TV (1920x1080)

Unidad	Duración	Herramientas	Puntos
U3 — PWA Smart TV	4-5 hrs	VS Code + Chrome DevTools + ffmpeg	100 pts

1. OBJETIVO

Crear desde cero un proyecto PWA correctamente estructurado para pantalla inteligente: configurar manifest.json, registrar un service worker con estrategia Cache First para recursos estáticos y Network First para datos de la API, emular una TV 1920x1080 en Chrome DevTools, y desplegar la primera versión con video de fondo según la condición climática.

1.1 Relación con practicas anteriores

Practica	Que aporta a P3.1
P2.3 — Weather model	El mismo modelo de datos Weather se reutiliza en la PWA
P2.5 — API OpenWeatherMap	La misma API key (en .env) y los mismos endpoints se usan aquí
P2.6 — BLE Monitor actividad	Los datos de actividad del wearable llegaran a la TV mas adelante (P3.3)
Figma P1.3 — Mockup TV	La UI de esta practica debe coincidir con el mockup de 1920x1080

CRITICO antes de empezar

Verifica que tu archivo .gitignore ya incluye: .env *.jks *.keystore node_modules/

La API key de OpenWeatherMap va en un archivo .env en la raíz del proyecto.

NUNCA la escribas directamente en app.js ni en ningún archivo que se versiona en Git.

Penalización: -50 pts irrecuperables si la API key aparece en cualquier commit.

2. PREPARACIÓN DEL ENTORNO

2.1 Herramientas necesarias

Herramienta	Versión mínima	Verificación	Descarga
VS Code	1.89+	code --versión	code.visualstudio.com
Google Chrome	120+	chrome://versión	google.com/chrome
Node.js	20+	node --versión	nodejs.org
Git	2.40+	git --versión	git-scm.com
ffmpeg	6.0+	ffmpeg -versión	ffmpeg.org
Live Server	VS Code extensión	Extensions: Live Server	VS Code Marketplace

2.2 Estructura del proyecto

Crea la carpeta del proyecto y la estructura de archivos completa:

Crear estructura del proyecto

```
mkdir clima-smart-tv
cd clima-smart-tv
git init

# Crear estructura de carpetas
mkdir -p assets/videos assets/posters icons css js

# Crear archivos base
touch index.html
touch css/styles.css
touch js/app.js
touch js/weather.js
touch js/navigation.js
touch manifest.json
touch sw.js
touch .env
touch .gitignore

# Verificar estructura
find . -not -path './.git/*' | sort
```

La estructura final debe verse así:

Estructura completa del proyecto

```

clima-smart-tv/
├─ index.html
├─ manifest.json
├─ sw.js
├─ .env                ← API key (NO en Git)
├─ .gitignore
├─ css/
│   └─ styles.css
├─ js/
│   ├── app.js         ← Punto de entrada, inicializa todo
│   ├── weather.js     ← Llamadas a la API
│   └─ navigation.js   ← Lógica D-pad
├─ assets/
│   ├── videos/        ← Videos de fondo por condición climática.
│   │   ├── clear.mp4
│   │   ├── cloudy.mp4
│   │   ├── rain.mp4
│   │   └─ thunder.mp4
│   └─ posters/        ← Imágenes estáticas como fallback
│       ├── clear.jpg
│       ├── cloudy.jpg
│       ├── rain.jpg
│       └─ thunder.jpg
└─ icons/
    ├── icon-192.png
    └─ icon-512.png

```

2.3 .gitignore y .env

Configura el .gitignore ANTES del primer commit:

.gitignore

```

# .gitignore
.env
node_modules/
*.jks
*.keystore
.DS_Store
Thumbs.db

# Verificar que .env no sera versionado:
# git status (no debe aparecer .env en la lista)

```

.env

```
# .env — NUNCA subir a Git
OPENWEATHER_API_KEY=tu_api_key_aqui
OPENWEATHER_BASE_URL=https://api.openweathermap.org/data/2.5/weather
```

Primer commit:

```
git add .
git status          # Verifica que .env NO aparece
git commit -m 'P3.1: Estructura inicial PWA clima-smart-tv'
git remote add origin https://github.com/tu-usuario/clima-smart-tv.git
git push -u origin main
```

3. MANIFEST.JSON Y METADATOS PWA

3.1 Crear manifest.json

El manifest define como se instala y presenta la PWA en el launcher de la TV:

manifest.json completo

```
{
  "name": "Clima Smart TV",
  "short_name": "Clima TV",
  "description": "Dashboard de clima en tiempo real para pantalla inteligente",
  "start_url": "/index.html",
  "display": "fullscreen",
  "orientation": "landscape",
  "background_color": "#0a0a1a",
  "theme_color": "#1a237e",
  "lang": "es-MX",
  "icons": [
    {
      "src": "/icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any maskable"
    },
    {
      "src": "/icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ],
  "categories": ["weather", "utilities"],
  "screenshots": [
    {
      "src": "/assets/posters/clear.jpg",
      "sizes": "1920x1080",
      "type": "image/jpeg",
      "form_factor": "wide"
    }
  ]
}
```

3.2 Vincularlo en index.html

Crea el archivo index.html con todos los meta tags necesarios:

index.html completo

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=1920" />
  <meta name="theme-color" content="#1a237e" />

  <!-- PWA -->
  <link rel="manifest" href="/manifest.json" />
  <link rel="apple-touch-icon" href="/icons/icon-192.png" />

  <!-- Content Security Policy -->
  <meta http-equiv="Content-Security-Policy"
    content="
      default-src 'self';
      script-src 'self';
      style-src 'self' 'unsafe-inline';
      img-src 'self' data: https://openweathermap.org;
      media-src 'self';
      connect-src 'self' https://api.openweathermap.org;
    "
  />

  <title>Clima Smart TV</title>
  <link rel="stylesheet" href="/css/styles.css" />
</head>
<body>

  <!-- Video de fondo (sin audio, loop automático) -->
  <video id="bgVideo" autoplay muted loop playsinline
    preload="auto" aria-hidden="true"
    poster="/assets/posters/clear.jpg">
    <source src="/assets/videos/clear.mp4" type="video/mp4" />
    <source src="/assets/videos/clear.webm" type="video/webm" />
    
  </video>

  <!-- Overlay oscuro para contraste sobre el video -->
  <div class="overlay"></div>
```

```
<!-- Header con ciudad y hora -->
<header class="tv-header">
  <span id="cityName">Cargando...</span>
  <span id="currentTime"></span>
</header>

<!-- Grid de 4 ciudades (2x2) -->
<main class="cities-grid" role="main">
  <article class="city-card focused" id="card0" tabindex="0"
    role="button" aria-label="Ciudad 1">
    <h2 class="city-name">--</h2>
    <p class="temperature">--°C</p>
    <p class="condition">--</p>
    <p class="details">Humedad: --% | Viento: -- m/s</p>
  </article>
  <article class="city-card" id="card1" tabindex="-1"
    role="button" aria-label="Ciudad 2">
    <h2 class="city-name">--</h2>
    <p class="temperature">--°C</p>
    <p class="condition">--</p>
    <p class="details">Humedad: --% | Viento: -- m/s</p>
  </article>
  <article class="city-card" id="card2" tabindex="-1"
    role="button" aria-label="Ciudad 3">
    <h2 class="city-name">--</h2>
    <p class="temperature">--°C</p>
    <p class="condition">--</p>
    <p class="details">Humedad: --% | Viento: -- m/s</p>
  </article>
  <article class="city-card" id="card3" tabindex="-1"
    role="button" aria-label="Ciudad 4">
    <h2 class="city-name">--</h2>
    <p class="temperature">--°C</p>
    <p class="condition">--</p>
    <p class="details">Humedad: --% | Viento: -- m/s</p>
  </article>
</main>

<script src="/js/weather.js"></script>
<script src="/js/navigation.js"></script>
<script src="/js/app.js"></script>
</body>
</html>
```


4. CSS — DISEÑO 10-FOOT PARA 1920x1080

Crea `css/styles.css` con el diseño completo para TV. Cada valor esta justificado para la regla de diseño 10-foot:

css/styles.css — diseño 10-foot completo

```
/* — Reset TV ————— */
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0; padding: 0;
}

:root {
  /* Tipografía 10-foot: mínimo 48px para datos primarios */
  --fs-temperature: 5rem;      /* 80px — dato principal */
  --fs-city:         2rem;      /* 32px — nombre ciudad */
  --fs-condition:    1.5rem;    /* 24px — condición */
  --fs-details:      1.25rem;   /* 20px — detalles */
  --fs-header:       1.75rem;   /* 28px — header */

  /* Safe zone 5% */
  --safe-v: 54px;    /* 5% de 1080 */
  --safe-h: 96px;    /* 5% de 1920 */

  /* Colores */
  --color-focus:    #FFD700;    /* dorado — foco visible */
  --color-bg:       rgba(0,0,0,0.55);
  --color-card:     rgba(255,255,255,0.12);
  --color-text:     #FFFFFF;
  --color-detail:   rgba(255,255,255,0.75);
}

/* — Layout base TV ————— */
html, body {
  width: 1920px;
  height: 1080px;
  overflow: hidden;      /* SIN scroll — todo visible */
  background: #0a0a1a;
  color: var(--color-text);
  font-family: 'Segoe UI', Arial, sans-serif;
}

/* — Video de fondo ————— */
#bgVideo {
```

```
position: fixed;
top: 0; left: 0;
width: 100%; height: 100%;
object-fit: cover;
z-index: -2;
}

.overlay {
position: fixed;
inset: 0;
background: rgba(0, 0, 0, 0.50);
z-index: -1;
}

/* — Header ————— */
.tv-header {
display: flex;
justify-content: space-between;
align-items: center;
padding: var(--safe-v) var(--safe-h) 24px;
font-size: var(--fs-header);
font-weight: 600;
letter-spacing: 0.05em;
text-shadow: 0 2px 8px rgba(0,0,0,0.8);
}

/* — Grid de ciudades ————— */
.cities-grid {
display: grid;
grid-template-columns: 1fr 1fr;
grid-template-rows: 1fr 1fr;
gap: 32px;
padding: 0 var(--safe-h) var(--safe-v);
height: calc(1080px - 54px - 80px); /* total - safe-v - header */
}

/* — Tarjeta ciudad ————— */
.city-card {
background: var(--color-card);
backdrop-filter: blur(12px);
border: 2px solid rgba(255,255,255,0.15);
border-radius: 20px;
padding: 40px 48px;
display: flex;
flex-direction: column;
```

```
justify-content: center;
gap: 12px;
cursor: pointer;
transition: transform 0.15s ease, border-color 0.15s ease;
outline: none;
}

/* — Estado de foco D-pad ————— */
.city-card.focused {
  border-color: var(--color-focus);
  box-shadow: 0 0 0 4px var(--color-focus),
              0 8px 32px rgba(255,215,0,0.3);
  transform: scale(1.02);
}

/* — Tipografía de tarjeta ————— */
.city-name    { font-size: var(--fs-city);      font-weight: 700; }
.temperature  { font-size: var(--fs-temperature);font-weight: 800; line-height:1; }
.condition    { font-size: var(--fs-condition); font-weight: 500; }
.details      { font-size: var(--fs-details);   color: var(--color-detail); }
```

5. JAVASCRIPT — CLIMA, NAVEGACIÓN Y VIDEO

5.1 weather.js — consumo de API

Crea js/weather.js. Nota: en un proyecto web puro la API key no puede ocultarse completamente del cliente. Para esta practica la leeremos desde una variable de entorno en un paso de build simple, o la declararemos como constante documentando que en producción iría en un backend proxy:

js/weather.js

```
// js/weather.js
// NOTA DE SEGURIDAD: En produccion, la API key debe ir en un
// backend proxy (Node/Express) que reciba peticiones del cliente
// y haga la llamada a OpenWeatherMap sin exponer la key.
// Para esta practica academica la declaramos como constante
// y documentamos que el archivo NO se sube a GitHub.

// En produccion: reemplazar por llamada a /api/weather?city=X
const API_KEY = window.ENV_API_KEY || 'TU_API_KEY_AQUI';
const BASE_URL = 'https://api.openweathermap.org/data/2.5/weather';

const CITIES = ['Queretaro', 'Ciudad de Mexico', 'Guadalajara', 'Monterrey'];

// Mapeo condicion -> archivo de video
const VIDEO_MAP = {
  Clear: { video:'/assets/videos/clear.mp4', poster:'/assets/posters/clear.jpg' },
  Clouds: { video:'/assets/videos/cloudy.mp4', poster:'/assets/posters/cloudy.jpg' },
  Rain: { video:'/assets/videos/rain.mp4', poster:'/assets/posters/rain.jpg' },
  Drizzle: { video:'/assets/videos/rain.mp4', poster:'/assets/posters/rain.jpg' },
  Thunderstorm: { video:'/assets/videos/thunder.mp4', poster:'/assets/posters/thunder.jpg' },
  Snow: { video:'/assets/videos/cloudy.mp4', poster:'/assets/posters/cloudy.jpg' },
};

async function fetchWeather(city) {
  // Sanitizar entrada
  const clean = city.trim().replace(/[^\w\s]/g, '');
  if (!clean) throw new Error('Ciudad invalida');

  const url = `${BASE_URL}?q=${encodeURIComponent(clean)}&appid=${API_KEY}
```

```

&units=metric&lang=es`;

const res = await fetch(url, { signal: AbortSignal.timeout(8000) });

if (!res.ok) {
  if (res.status === 401) throw new Error('API key invalida');
  if (res.status === 404) throw new Error(`Ciudad '${city}' no encontrada`);
  throw new Error(`Error API: ${res.status}`);
}

const json = await res.json();

// Validar estructura antes de usar
if (!json.main || !json.weather?.[0]) {
  throw new Error('Respuesta de API inesperada');
}

return {
  city:      json.name,
  temperature: Math.round(json.main.temp),
  condition:  json.weather[0].main,
  description: json.weather[0].description,
  humidity:   json.main.humidity,
  windSpeed:  json.wind?.speed?.toFixed(1) ?? '0',
};
}

async function fetchAllCities() {
  const results = await Promise.allSettled(
    CITIES.map(city => fetchWeather(city))
  );
  return results.map((r, i) =>
    r.status === 'fulfilled'
      ? r.value
      : { city: CITIES[i], temperature: '--', condition: 'Error',
        description: 'Sin datos', humidity: '--', windSpeed: '--' }
  );
}

```

5.2 navigation.js — D-pad

js/navigation.js

```
// js/navigation.js
```

```
const NAV_MAP = {
  card0: { right:'card1', down:'card2' },
  card1: { left:'card0', down:'card3' },
  card2: { right:'card3', up:'card0' },
  card3: { left:'card2', up:'card1' },
};

let currentFocus = 'card0';

function moveFocus(nextId) {
  document.getElementById(currentFocus).classList.remove('focused');
  document.getElementById(currentFocus).setAttribute('tabindex', '-1');
  currentFocus = nextId;
  const el = document.getElementById(currentFocus);
  el.classList.add('focused');
  el.setAttribute('tabindex', '0');
  el.focus({ preventScroll: true });
}

document.addEventListener('keydown', e => {
  const dir = {
    ArrowRight:'right', ArrowLeft:'left',
    ArrowDown:'down',   ArrowUp:'up',
  }[e.key];

  if (dir) {
    e.preventDefault();
    const next = NAV_MAP[currentFocus]?.[dir];
    if (next) moveFocus(next);
    return;
  }

  // Enter / OK del control remoto - seleccionar tarjeta
  if (e.key === 'Enter' || e.key === ' ') {
    e.preventDefault();
    const card = document.getElementById(currentFocus);
    card.dispatchEvent(new CustomEvent('card-select', {
      bubbles: true,
      detail: { cardId: currentFocus }
    }));
  }
});
```

5.3 app.js — orquestación principal

js/app.js

```
// js/app.js

// — Reloj en tiempo real —————
function updateClock() {
  const now = new Date();
  document.getElementById('currentTime').textContent =
    now.toLocaleTimeString('es-MX', { hour:'2-digit', minute:'2-digit' });
}
setInterval(updateClock, 1000);
updateClock();

// — Cambiar video de fondo segun condicion —————
function updateBackground(condition) {
  const map = VIDEO_MAP[condition] || VIDEO_MAP['Clouds'];
  const video = document.getElementById('bgVideo');

  if (video.querySelector('source').src.includes(map.video)) return;

  video.style.opacity = '0';
  video.poster = map.poster;
  video.querySelector('source[type="video/mp4"]').src = map.video;
  video.load();
  video.play().catch(() => {
    // Autoplay bloqueado - mostrar poster como fallback
    document.body.style.backgroundImage = `url(${map.poster})`;
  });
  video.style.opacity = '1';
}

// — Renderizar tarjeta —————
function renderCard(cardId, data) {
  const card = document.getElementById(cardId);
  card.querySelector('.city-name').textContent = data.city;
  card.querySelector('.temperature').textContent = `${data.temperature}°C`;
  card.querySelector('.condition').textContent = data.description;
  card.querySelector('.details').textContent =
    `Humedad: ${data.humidity}% | Viento: ${data.windSpeed} m/s`;
}

// — Seleccionar ciudad con Enter —————
document.addEventListener('card-select', e => {
```



```
const idx = parseInt(e.detail.cardId.replace('card', ''));
if (window._weatherData?.[idx]) {
  const data = window._weatherData[idx];
  updateBackground(data.condition);
  document.getElementById('cityName').textContent = data.city;
}
});

// — Cargar datos al iniciar —————
async function init() {
  try {
    const data = await fetchAllCities();
    window._weatherData = data;

    data.forEach((d, i) => renderCard(`card${i}`, d));

    // Video de fondo segun la primera ciudad
    updateBackground(data[0].condition);
    document.getElementById('cityName').textContent = data[0].city;

  } catch (err) {
    console.error('Error cargando clima:', err.message);
    document.getElementById('cityName').textContent = 'Error de conexion';
  }
}

// — Registrar Service Worker —————
if ('serviceWorker' in navigator) {
  navigator.serviceWorker.register('/sw.js')
    .then(reg => console.log('SW registrado:', reg.scope))
    .catch(err => console.error('SW error:', err));
}

init();

// Refrescar datos cada 10 minutos
setInterval(init, 10 * 60 * 1000);
```

6. SERVICE WORKER

Crea `sw.js` con estrategias de cache diferenciadas por tipo de recurso:

sw.js — Service Worker con estrategias

```
// sw.js
const CACHE_STATIC = 'clima-tv-static-v1';
const CACHE_DYNAMIC = 'clima-tv-dynamic-v1';

// Recursos estaticos: se cachean en la instalacion
const STATIC_ASSETS = [
  '/',
  '/index.html',
  '/css/styles.css',
  '/js/app.js',
  '/js/weather.js',
  '/js/navigation.js',
  '/manifest.json',
  '/icons/icon-192.png',
  '/icons/icon-512.png',
  '/assets/posters/clear.jpg',
  '/assets/posters/cloudy.jpg',
  '/assets/posters/rain.jpg',
  '/assets/posters/thunder.jpg',
];

// — Instalacion: cachear estaticos —————
self.addEventListener('install', e => {
  e.waitUntil(
    caches.open(CACHE_STATIC)
      .then(cache => cache.addAll(STATIC_ASSETS))
      .then(() => self.skipWaiting())
  );
});

// — Activacion: limpiar caches viejos —————
self.addEventListener('activate', e => {
  e.waitUntil(
    caches.keys().then(keys =>
      Promise.all(
        keys
          .filter(k => k !== CACHE_STATIC && k !== CACHE_DYNAMIC)
          .map(k => caches.delete(k))
      )
    )
  );
});
```

```
    ).then(() => self.clients.claim())
  );
});

// — Fetch: estrategia segun tipo de recurso —————
self.addEventListener('fetch', e => {
  const { request } = e;
  const url = new URL(request.url);

  // API de clima: Network First (datos frescos)
  if (url.hostname === 'api.openweathermap.org') {
    e.respondWith(networkFirst(request));
    return;
  }

  // Videos: Cache First (muy pesados, no cambian)
  if (request.url.includes('/assets/videos/')) {
    e.respondWith(cacheFirst(request));
    return;
  }

  // Todo lo demas: Cache First (estaticos)
  e.respondWith(cacheFirst(request));
});

async function cacheFirst(request) {
  const cached = await caches.match(request);
  if (cached) return cached;
  const response = await fetch(request);
  const cache = await caches.open(CACHE_DYNAMIC);
  cache.put(request, response.clone());
  return response;
}

async function networkFirst(request) {
  try {
    const response = await fetch(request,
      { signal: AbortSignal.timeout(5000) });
    const cache = await caches.open(CACHE_DYNAMIC);
    cache.put(request, response.clone());
    return response;
  } catch {
    const cached = await caches.match(request);
    return cached ?? new Response(JSON.stringify({ error: 'Sin conexion' }),
      { headers: { 'Content-Type': 'application/json' } });
  }
}
```

```
}  
}
```

7. EMULACIÓN TV EN CHROME DEVTOOLS

7.1 Configurar emulación 1920x1080

1. Abre la PWA en Chrome con Live Server: clic derecho en index.html -> Open with Live Server. Se abrirá en `http://127.0.0.1:5500`.
2. Abre DevTools: F12 (Windows/Linux) o Cmd+Option+I (macOS).
3. Activa modo dispositivo: icono de telefono/tablet en la barra de DevTools, o Ctrl+Shift+M.
4. En el selector de dispositivos escoge Edit... o Agregar dispositivo personalizado:

Campo	Valor
Device name	Smart TV 1080p
Width	1920
Height	1080
Device pixel ratio	1
User agent string	Mozilla/5.0 (SMART-TV; Linux; Tizen 6.0) AppleWebKit/538.1

5. Selecciona Smart TV 1080p en el selector. La pantalla debe mostrar la PWA en 1920x1080 sin scrollbars.
6. Verifica visualmente:
 - ☐ Ningún elemento toca el borde — safe zone activa.
 - ☐ Temperatura visible y legible desde lejos (5rem = 80px).
 - ☐ Cuatro tarjetas ocupan el espacio sin overflow.
 - ☐ El video de fondo ocupa toda la pantalla.

7.2 Verificar el Service Worker

En DevTools: Application -> Service Workers. Verifica que el SW esta en estado Active and running. En Application -> Cache Storage verifica que clima-tv-static-v1 contiene todos los archivos estáticos.

Prueba offline: activa el checkbox Offline en la sección Service Workers. Recarga la pagina. La PWA debe seguir cargando con datos del cache.

8. PREPARAR VIDEOS DE FONDO

8.1 Opción A — Videos propios con ffmpeg

Si tienes archivos de video propios, optimízalos para web:

Convertir videos con ffmpeg

```
# Convertir a MP4 H.264 optimizado para web TV
ffmpeg -i tu_video_lluvia.mp4 \
  -vcodec libx264 -crf 23 -preset slow \
  -vf scale=1920:1080 -r 30 -an \
  -movflags +faststart \
  assets/videos/rain.mp4

# Generar poster (primera frame como imagen)
ffmpeg -i assets/videos/rain.mp4 -vframes 1 -q:v 2 \
  assets/posters/rain.jpg
```

8.2 Opción B — Videos de Pixabay (libres de derechos)

Descarga videos libres de derechos de estas fuentes. Busca: 'sky clear loop', 'rain loop 4k', 'storm lightning loop':

- [Pixabay.com/videos](https://pixabay.com/videos/) — filtrar: free, 1080p, loop
- [Pexels.com/videos](https://pexels.com/videos/) — filtrar: landscape, nature, 1080p
- [Coverr.co](https://coverr.co) — videos de fondo para web, ya optimizados

Requisito mínimo para esta practica

Necesitas al menos 2 videos para poder demostrar el cambio según condición climática:

- clear.mp4 (cielo despejado, azul)
- rain.mp4 (lluvia o nubes oscuras)

Si no tienes acceso a video, usa posters .jpg como fallback en el <video>.

9. CHECKLIST Y RUBRICA

Lista de verificación

- ☐ Estructura de carpetas correcta (js/, css/, assets/, icons/)
- ☐ .gitignore incluye .env antes del primer commit
- ☐ API key NO aparece en ningun archivo versionado (verificar con git log -S 'API')
- ☐ manifest.json valido con name, short_name, display, icons (192 y 512)
- ☐ Service worker registrado y activo (DevTools -> Application -> SW)
- ☐ Cache estatico poblado en DevTools -> Cache Storage
- ☐ App visible en emulador 1920x1080 sin scrollbars
- ☐ Safe zone activa: ningun elemento toca el borde de la pantalla
- ☐ Tipografia: temperatura >= 5rem, ciudad >= 2rem
- ☐ Cuatro tarjetas de ciudades con datos reales de la API
- ☐ Navegacion D-pad funciona: flechas cambian el foco con borde dorado
- ☐ Video de fondo cambia al presionar Enter en cada tarjeta
- ☐ Modo offline: app carga desde cache cuando se activa Offline en DevTools
- ☐ Commit: 'P3.1: PWA clima-smart-tv configuración inicial'

Rubrica de evaluación

Criterio	Descripción	Puntos
Estructura y Git	Estructura correcta, .gitignore antes del commit, sin API key en repo	15
manifest.json	Todos los campos requeridos, iconos 192 y 512, display fullscreen	15
Service Worker	Registrado y activo, estrategias Cache First / Network First correctas	20
Modo offline	App carga desde cache con Offline activado en DevTools	10
Layout 1920x1080	Sin scroll, safe zone activa, grid 2x2 ocupando el espacio	15
Datos API reales	4 ciudades con temperatura, condicion, humedad y viento	15
Video de fondo	Cambia segun condicion climatica, con fallback a poster si falla	10

TOTAL: 100 puntos

Siguiente practica: P3.2

P3.2 construye sobre esta base: agrega D-pad completo, multimedia avanzado y datos de actividad del wearable. Asegurate de que P3.1 este 100% funcional antes de continuar. El repositorio de P3.1 es el mismo para P3.2, P3.3 y P3.4.